



ALGORITMOS E ESTRUTURA DE DADOS

Virgínia Mara Cardoso

ALGORITMOS E ESTRUTURA DE DADOS

1ª edição

Londrina
Editora e Distribuidora Educacional S.A.
2023

2023 por Editora e Distribuidora Educacional S.A.

Todos os direitos reservados. Nenhuma parte desta publicação poderá ser reproduzida ou transmitida de qualquer modo ou por qualquer outro meio, eletrônico ou mecânico, incluindo fotocópia, gravação ou qualquer outro tipo de sistema de armazenamento e transmissão de informação, sem prévia autorização, por escrito, da Editora e Distribuidora Educacional S.A.

Diretora Sr. de Pós-graduação & OPM

Silvia Rodrigues Cima Bizatto

Conselho Acadêmico

Alessandra Cristina Fahl
Ana Carolina Gulelmo Staut
Camila Braga de Oliveira Higa
Camila Turchetti Bacan Gabiatti
Giani Vendramel de Oliveira
Gislaine Denisale Ferreira
Henrique Salustiano Silva
Mariana Gerardi Mello
Nirse Ruscheinsky Breternitz
Priscila Pereira Silva

Coordenador

Henrique Salustiano Silva

Revisor

Rafael Albuquerque Pinto

Editorial

Beatriz Meloni Montefusco
Carolina Yaly
Márcia Regina Silva
Paola Andressa Machado Leal

Dados Internacionais de Catalogação na Publicação (CIP)

C268a Cardoso, Virgínia Mara
Algoritmos e estrutura de dados/ Virgínia Mara Cardoso, –
Londrina: Editora e Distribuidora Educacional S.A., 2023.
32 p.

ISBN 978-65-5903-346-1

1. Algoritmo. 2. Estrutura. 3. Dados. I. Título.

CDD 005.8

Raquel Torres – CRB 8/10534

ALGORITMOS E ESTRUTURA DE DADOS

SUMÁRIO

Apresentação da disciplina	05
Introdução aos algoritmos e estrutura de dados	07
Tipos abstratos de dados	19
A estrutura de dados Árvores	31
Estrutura de Dados Avançadas	43



Apresentação da disciplina

Aluno, se você deseja compreender a base da programação, a disciplina de Algoritmo e Estrutura de Dados é fundamental para seu aprendizado. Essa disciplina ensina como estruturar e manipular dados de forma eficiente, bem como desenvolver algoritmos eficazes para resolver problemas computacionais.

Os algoritmos, sequências de instruções bem definidas, são usados para resolver problemas computacionais de forma sistemática e eficiente. Eles são a base para a maioria dos sistemas de software modernos e podem ser aplicados em uma ampla variedade de áreas, desde a computação até a inteligência artificial.

As estruturas de dados, por sua vez, são responsáveis por organizar e manipular dados de forma eficiente, permitindo que os algoritmos resolvam problemas computacionais com rapidez e precisão. Ao longo desta disciplina, você irá aprender as principais estrutura de dados, incluindo filas, listas, pilhas, árvores e grafos.

As filas, por exemplo, são estruturas de dados que permitem a adição de novos elementos em uma extremidade e a remoção de elementos da outra extremidade, sendo frequentemente usadas em situações em que é necessário processar elementos em uma ordem específica, como em sistemas de gerenciamento de pedidos ou em filas de atendimento.

Já as listas são estruturas de dados que permitem o armazenamento de um conjunto de elementos, permitindo a adição e remoção de elementos de forma flexível, sendo frequentemente usadas em algoritmos de busca e ordenação.

As pilhas, por sua vez, são estruturas de dados semelhantes às filas, usadas em algoritmos de *backtracking* e em sistemas de processamento de expressões matemáticas.

As árvores, estruturas de dados que representam uma hierarquia de elementos, são usadas em algoritmos de busca, como a busca binária, e em sistemas de organização de dados hierárquicos.

E, por fim, você irá aprender sobre estruturas de dados avançadas, como os grafos, que representam uma rede de conexões entre elementos e são frequentemente usados em algoritmos de otimização e em sistemas de análise de redes sociais e de transportes.

Nesta disciplina, você irá aprender a projetar, implementar e analisar algoritmos eficazes e estruturas de dados que possam ser aplicados a uma ampla variedade de problemas computacionais.

Ao final do curso você terá adquirido uma compreensão sólida dos conceitos fundamentais de algoritmos e estrutura de dados, bem como habilidades práticas para aplicar esses conceitos na resolução de problemas reais. Isso o preparará para continuar a desenvolver suas habilidades de programação e se tornar um programador mais eficiente e bem-sucedido.

Introdução aos algoritmos e estrutura de dados

Autoria: Virgínia Mara Cardoso

Leitura crítica: Rafael Albuquerque Pinto



Objetivos

- Apresentar os algoritmos e suas representações.
- Introduzir os conceitos de estruturas de dados.
- Introduzir a análise de algoritmos e a complexidade assintótica.
- Conhecer alocação dinâmica, ponteiros e arquivo.



1. Introdução

Nesta aula vamos tratar de um assunto muito pertinente para programadores e para aqueles que trabalham com dados ou desejam conhecer e aprender um pouco mais sobre os algoritmos.

Para começarmos, abordaremos os algoritmos e suas representações. Inicialmente, discutiremos os conceitos de estruturas de dados, que constituem o foco principal desta aula. Explicaremos o que são essas estruturas e por que são utilizadas em programação.

Além disso, apresentaremos a você uma introdução à análise de algoritmos, que explicará como é possível melhorar o desempenho desses algoritmos. Para concluir, mostraremos recursos que podem ser utilizados para trabalhar com dados de forma eficiente.



2. Conceitos de algoritmos e suas representações

Nesta aula vamos ver o que são os algoritmos, como podemos representá-los e a sua utilização. Além disso, iremos entender os algoritmos e os programas de computadores.

No mundo da tecnologia, é bastante comum o uso da palavra “algoritmo”. Essa palavra tem sua origem no sobrenome de um famoso matemático árabe do século IX, chamado Al-Khwarizmi (SOUSA, [s. d.]).

Na literatura, podemos encontrar diversas definições de algoritmos, afinal esse conceito existe há séculos e é amplamente utilizado em áreas como matemática e ciência da computação. Podemos defini-lo como uma sequência de passos que seguimos para realizar uma determinada tarefa. São instruções determinadas para resolver um problema específico.

Agora, você pode estar se perguntando se os programas de computador, que consistem em sequências de códigos, estão relacionados aos algoritmos. A resposta é sim, afirmamos que um programa de computador é um algoritmo, pois consiste em uma sequência de códigos escritos em uma linguagem específica para resolver um determinado problema. Segundo Cormen (2002), um algoritmo é definido como um “procedimento computacional bem definido que recebe um ou mais valores como entrada e produz um ou mais valores como saída”. Em outras palavras, o algoritmo é um conjunto de instruções que, quando executado corretamente, produz um resultado desejado. Esse resultado pode ser a solução de um problema, a realização de uma tarefa ou a obtenção de um resultado específico.

Podemos encontrar exemplos de algoritmos em nossas tarefas diárias mais simples. Imagine que você vá fazer um brownie, uma torta ou mesmo um simples café, é necessário você seguir uma receita, separar os ingredientes, ir colocando-os na ordem correta e depois colocar pra assar ou, no caso do café, filtrar. Para que você tenha o brownie, o bolo ou o café prontos para servir, todas essas ações devem seguir uma sequência ordenada, caso contrário, não conseguimos resolver a questão.

No entanto, você pode se perguntar se todas as ações podem ser consideradas algoritmos. A resposta é não. Para algo ser considerado um algoritmo, é preciso que haja uma sequência finita de instruções bem definidas que satisfaçam as seguintes características:

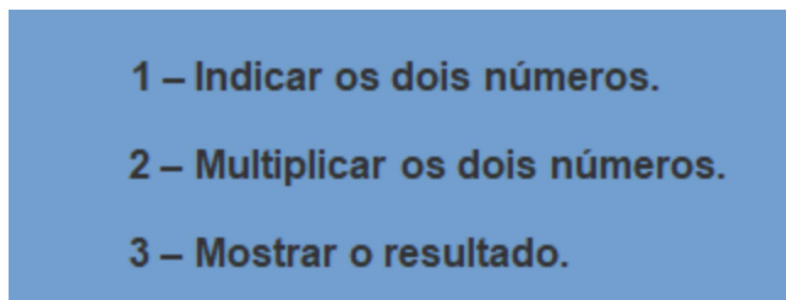
- Uma entrada com um valor ou um conjunto de valores.
- Processamento, instruções bem claras.
- Uma saída, um valor ou um conjunto de valores, como resultado.
- Uma sequência finita de passos.

Temos várias maneiras de representar um algoritmo (MANZANO,2015), dentre as mais comuns podemos citar três:

- Descrição narrativa;
- Pseudocódigo;
- Fluxograma.

A descrição narrativa é uma forma de apresentar a sequência de passos utilizando uma linguagem natural, conforme descreve Manzano (2015). Imagine que você vá multiplicar dois números, como seria o algoritmo em uma descrição narrativa? Como usamos uma linguagem natural, e deve ser uma sequência ordenada de passos, teríamos, como indica a Figura 1:

Figura 1 – Representação do algoritmo em descrição narrativa



Fonte: elaborada pelo autor.

Você deve estar notando que a descrição narrativa é bem simples, uma linguagem direta e objetiva.

A outra forma de representação do algoritmo é o pseudocódigo. Como o nome indica, utilizamos uma linguagem codificada (MANZANO, 2015), seguimos regras determinadas, predefinidas. Essas regras são estabelecidas para seguirmos um padrão próximo ao utilizado nas linguagens de programação. Utilizando o mesmo exemplo de multiplicação de dois números, teríamos a representação em pseudocódigo, conforme a Figura 2.

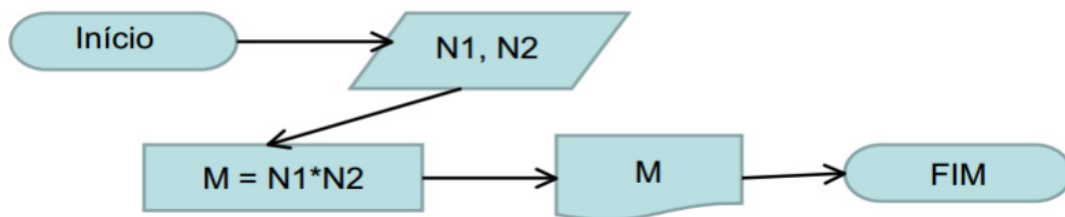
Figura 2 – Representação do algoritmo em pseudocódigo



Fonte: elaborada pelo autor.

Segundo Manzano (2015), na representação do fluxograma utilizamos símbolos gráficos para a representação, temos os símbolos específicos para cada ação. O fluxograma é muito utilizado, pois a visualização fica bem clara e com muitos recursos. Podemos ver, na Figura 3, o exemplo da multiplicação de dois números em um fluxograma:

Figura 3 – Representação do algoritmo em fluxograma



Fonte: elaborada pelo autor.

Aqui deixamos uma indicação de ferramenta muito interessante de algoritmos, para você praticar um pouco. Essa ferramenta pode ser executada no Windows, Linux ou MacOS. É a Portugol Studio.

A seguir, vamos aprofundar nosso entendimento sobre como trabalhar com algoritmos, explorando os recursos que podemos utilizar e iniciando nosso aprendizado sobre dados e suas estruturas.



3. Introdução à estrutura de dados

Na área da computação, utilizamos algoritmos como programas para processar dados e informações. Embora existam diversas definições de programas, de forma geral, podemos entendê-los como algoritmos expressos em códigos de uma linguagem formal, nos quais todas as instruções e etapas são executadas.

Para trabalhar com os dados existem estudos, técnicas para organizá-los, e a maneira de organizar os dados e operar sobre eles é o que denominamos de estrutura de dados. Baseado em Szwarcfiter e Markenzon (2020) e Pinto *et al.* (2020) é conceituado e explanado sobre as estruturas de dados.

Na programação, os dados são representados por meio de estruturas que chamamos de variáveis ou constantes. As variáveis são usadas para armazenar dados temporários, enquanto as constantes são utilizadas para armazenar dados estáticos, ou seja, que não podem ser alterados ao longo da execução do programa.

Para armazenar dados em programas de computador, é necessário classificá-los de acordo com seu tipo. Os tipos mais comuns incluem:

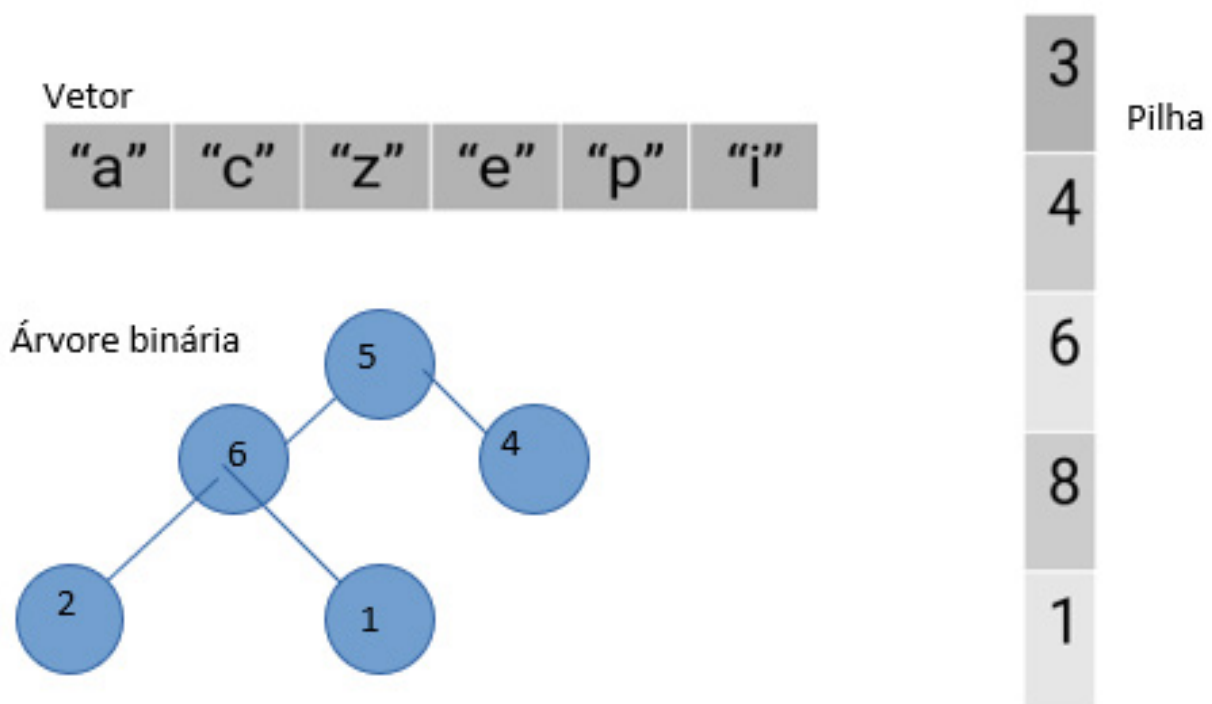
- Simples: os grupos de valores indivisíveis. Ex.: inteiro, *double* e *boolean*.
- Estruturados: um conjunto de valores simples ou um conjunto de valores de tipos diferentes.

Assim, quando mencionamos estrutura de dados, estamos falando sobre um modo de armazenar e organizar os dados para que possam ser utilizados na programação de uma forma mais eficiente. Assim podemos dizer que o objetivo das estruturas de dados é ter uma

organização para a manipulação e armazenamento dos dados na execução do programa.

Conforme mencionado anteriormente, existem dados simples e estruturados, assim como estruturas para lidar com ambos. As estruturas de dados simples incluem matrizes e vetores, permitindo o armazenamento de dados de um único tipo, e o acesso pode ser direto ou sequencial. Já as estruturas de dados complexas são utilizadas para representar conjuntos de dados mais específicos, tais como listas, filas, árvores e grafos. Na Figura 4, é possível observar alguns exemplos de estruturas simples e estruturadas, evidenciando como é mais fácil trabalhar com dados organizados, além de destacar a importância dessas estruturas para um algoritmo mais eficiente.

Figura 4 – Exemplos de estruturas de dados



Fonte: elaborada pelo autor.

Até aqui, você pôde aprender sobre algoritmos e como a estrutura de dados está relacionada a eles. Agora, vamos explorar como os algoritmos são analisados.



4. Análise de algoritmos e complexidade assintótica

Na área da computação, busca-se constantemente melhorar a performance dos computadores, e isso inclui os algoritmos. Surge então a pergunta: como podemos melhorar os algoritmos? E como eles são analisados? Ao criar um algoritmo, é essencial verificar sua corretude, mas também é importante analisar sua eficiência e tempo de execução.

Dizemos que o algoritmo está correto se temos o resultado esperado e se eles executam em tempo e espaço esperados dizemos que são eficientes. Para realizar a análise dos algoritmos, é preciso levar em consideração os dados de entrada e a sua influência no desempenho do algoritmo. Além disso, é necessário avaliar a influência da representação dos dados. O objetivo da análise de algoritmo é verificar se o algoritmo é capaz de resolver o problema em questão e qual é o seu desempenho, ou seja, quanto tempo ele leva para processar uma entrada considerando o seu tamanho. Donald Knuth é um cientista da computação americano conhecido por suas contribuições para a análise de algoritmo. Ele desenvolveu técnicas para prever o tempo de execução e o consumo de espaço de um algoritmo (KNUTH, 2022). Mas além da análise do desempenho e corretude, também são estudados alguns paradigmas que são úteis na elaboração de algoritmos para vários problemas computacionais. A análise é importante também no sentido de poder prever os recursos que o algoritmo precisará. Para avaliar um algoritmo, é importante levar em consideração critérios como sua correção, eficiência temporal e espacial. Muitas vezes, é utilizado o estudo da complexidade assintótica, que analisa o comportamento do algoritmo para um volume muito grande de dados de entrada, por meio de funções de custo.

A análise assintótica é um método que utiliza cálculos de função matemática com valores tendendo ao infinito e notações específicas para estudar o algoritmo e entender sua complexidade. Por meio dos gráficos

gerados, é possível obter uma visão mais precisa e identificar possíveis melhorias no algoritmo. Para compreender melhor essa análise, é importante conhecer alguns recursos que são frequentemente utilizados.

5. Alocação dinâmica, ponteiros e arquivos

No item anterior, mencionamos que às vezes precisamos lidar com algoritmos que envolvem grandes volumes de dados, cuja proporção não conhecemos de antemão, e que é importante garantir a eficiência do algoritmo. Neste item, vamos discutir alguns recursos que podemos usar para lidar melhor com esses dados e tornar o algoritmo mais eficiente e correto.

É fundamental considerar o uso de memória, mas como podemos alocar memória sem saber quanto iremos precisar? Dependendo da linguagem de programação utilizada, a quantidade de memória necessária pode ser conhecida. Existem comandos que permitem essa alocação, porém, estamos falando de algoritmos de maneira genérica, sem especificar uma linguagem de programação específica. Quando nos deparamos com esse tipo de problema, em que a quantidade de memória necessária só é conhecida durante a execução do programa, podemos recorrer à alocação dinâmica de memória, um recurso que evita o desperdício de memória (CORMEN, 2002).

Suponha que você precise usar uma matriz e declare no início do seu algoritmo que ela deve ter 2000 linhas e 30 colunas, mas durante a execução, você acaba usando apenas um terço da capacidade. Todo o espaço foi alocado e isso impacta na memória, no tempo de processamento e em outros detalhes, dependendo do algoritmo. Nesse caso, fica claro que você não utilizou o recurso correto, que seria a alocação dinâmica, evitando desperdício de memória. Portanto, quando você não consegue estimar a quantidade de memória que será

necessária, é preciso utilizar a alocação dinâmica, permitindo que você crie variáveis e estruturas de dados em tempo de execução.

Aqui você já viu que estamos começando a falar de recursos que, para utilizar, precisamos recorrer a estruturas de dados mais complexas. Lembrando que estruturas simples são vetores e matrizes, nessas estruturas armazenamos um único tipo de dado. No entanto, quando você trabalha com estruturas mais complexas, como fila, lista, grafo, entre outras, precisamos utilizar elementos como a alocação dinâmica, ponteiros e arquivos.

A alocação dinâmica e os ponteiros são elementos fundamentais para trabalhar com estruturas de dados mais complexas. E aqui você pergunta: “Mas o que são ponteiros?”. Segundo Manzano (2015), são variáveis compostas que permitem armazenar o endereço de memória. Podemos ver que o conteúdo da variável ponteiro difere das demais, pois aqui é o endereço de memória, que indica onde as informações estão e não o que está armazenado, como mostra a Figura 5.

Figura 5 – Exemplo de variáveis na memória

Endereço	Conteúdo
0xFF5	
0xFF4	78
0xFF3	
0xFF2	
0xFF1	33
0xFF0	

Conteúdo da variável : 78

Endereço da variável : 0xFF4

Conteúdo da variável ponteiro: 0xFF4

No que diz respeito ao ponteiro, você pode utilizá-lo quando trabalha com matrizes para manipular seus elementos, mudar valores das funções, manipular arquivos, aumentar a eficiência de algumas rotinas, trabalhar com estruturas de dados mais complexas.

Também temos o recurso dos arquivos, que são estruturas que possibilitam o armazenamento de dados externos em memória secundária. Essa abordagem apresenta vantagens e desvantagens. Por um lado, proporciona um espaço maior de armazenamento, porém, por outro lado, o acesso aos dados é mais lento. Apesar disso, o uso de arquivos é fundamental, pois eles permitem a preservação das informações mesmo quando o programa não está em execução.

No arquivo a informação fica armazenada em memória secundária, podendo ser o *hard disk*, a fita magnética, o CD/DVD, o pen drive, cartões, entre outros.

Temos arquivos textos, para armazenar textos e binários que usamos para armazenar dados ou objetos. Os arquivos são muito utilizados para os backups.



6. Conclusão

Nesta aula você pôde ter uma introdução sobre esta relevante e imprescindível disciplina *Algoritmos e Estrutura de Dados*. Foram apresentados os principais conceitos de uma forma clara e direta, para que você possa contextualizar rapidamente.



Referências

CORMEN, T. H. *et al.* **Algoritmos:** teoria e prática. Rio de Janeiro: Elsevier, 2002.

KNUTH, D. E. **The Art of Computer Programming: Combinatorial Algorithms**. Addison-Wesley Professional, 2022.

MANZANO, J. A. N. G. **Algoritmos: técnicas de programação**. 2. ed. São Paulo: Érica, 2015.

PINTO, R. A. *et al.* **Estrutura de dados**. Porto Alegre: SAGAH, 2020.

SOUSA, S. M. **Introdução à Teoria da Computação**. Disponível em: <https://www.di.ubi.pt/~desousa/TC/apresentacao-intro-text.pdf>. Acesso em: 25 jan. 2023.

SZWARCFITER, J. L.; MARKENZON, L. **Estruturas de dados e seus algoritmos**. Rio de Janeiro: LTC, 2020.

Tipos abstratos de dados

Autoria: Virgínia Mara Cardoso

Leitura crítica: Rafael Albuquerque Pinto



Objetivos

- Compreender o conceito de Tipos Abstratos de Dados (TADs).
- Explorar os fundamentos de listas e suas operações.
- Apresentar as características das pilhas, incluindo suas operações.
- Introduzir os conceitos básicos de filas, incluindo suas operações.



1. Introdução

Nesta aula vamos conhecer um modelo matemático para trabalharmos com as estruturas de dados. Quando precisamos desenvolver algoritmos mais complexos, especialmente para lidar com grandes volumes de dados, as estruturas de dados são recursos valiosos. Dentro dessas estruturas, os Tipos Abstratos de Dados (TADs) desempenham um papel crucial, fornecendo uma abstração para a manipulação de dados. Nesta aula, você terá a oportunidade de aprender sobre TADs, incluindo como e quando utilizá-los. Além disso, apresentaremos exemplos de filas, listas e pilhas, e você aprenderá suas operações e como manipulá-los para resolver problemas em programação.



2. Tipos abstratos de dados

Quando trabalhamos com dados, que, aliás, é a maior parte do tempo para quem programa, enfrentamos um grande desafio para lidar com eles. Temos vários tipos de dados e o volume de dados pode ser grande, dentre outros fatores. Nesse contexto, precisamos criar algoritmos corretos, confiáveis e rápidos, ou seja, com um desempenho que atenda às necessidades do problema. Para termos algoritmos mais eficientes e rápidos, precisamos saber como trabalhar de forma adequada. Quando estudamos estrutura de dados na computação, aprendemos técnicas e métodos que podemos utilizar para facilitar e indicar recursos que nos auxiliem a lidar com os dados.

Vou explicar como trabalhamos com estruturas e métodos. Um importante conceito é o de TADs, ou Tipos Abstratos de Dados. Mas o que isso significa? Será que é um outro tipo de dado que devemos utilizar? Não. Segundo Szwarcfiter e Markenzon (2020), TADs são conjuntos de dados encapsulados com seus procedimentos. Alguns autores se referem a eles como um modelo matemático. Mas

como isso funciona na prática? Imagine que você tenha um modelo matemático no qual trabalha exclusivamente com os dados do conjunto, ou seja, eles estão encapsulados e têm operações exclusivas para manipulá-los.

Vamos explorar mais detalhadamente como funciona esse conjunto de dados e seus procedimentos.

Ao pensar na implementação de um programa, você pode ter em mente que os procedimentos se referem a funções, subprogramas ou métodos. Em TADs, temos justamente essas duas partes: os dados encapsulados e as funções para manipulá-los. Os dados encapsulados são armazenados de forma a permitir que apenas as funções específicas para manipulá-los tenham acesso a eles. Isso torna o código mais organizado e mais fácil de ser avaliado, contribuindo para a confiabilidade e correção do programa. Já sabemos o que são TADs, mas agora surge a questão: como trabalhar com eles? Quando um programa é baseado em TADs, você estará lidando com dados encapsulados e funções específicas projetadas para manipular esses dados.

Podemos perceber que, ao utilizar TADs, teremos subprogramas que permitem escrever um código mais conciso e facilitam a correção de erros, resultando em um programa mais preciso. As funções exclusivas para manipular esses dados também tornam o código mais fácil de ser avaliado e confiável, o que garante menos erros e maior confiabilidade, facilitando a avaliação do código como um todo.

No entanto, você pode estar se perguntando: “Como isso afeta o meu programa?” Se você estiver utilizando TADs, é necessário ter duas partes:

- Definição das estruturas de dados.
- Definição dos procedimentos, ou seja, as funções que devem manipular os dados.

Para manipular os dados em TADs, não é necessário armazená-los em vetores ou matrizes, como usualmente fazemos em programas. Ao contrário, os dados são armazenados internamente em estruturas encapsuladas, e as operações são realizadas por meio de funções específicas para manipulá-los. Segundo Manzano (2015), essas funções devem ser cuidadosamente projetadas para inserir, excluir, alterar ou consultar os dados, levando em consideração as particularidades de cada estrutura.

Existem diversos TADs comuns e amplamente utilizados, tais como listas, pilhas e filas. É interessante conhecer essas estruturas, e agora vamos aprender como trabalhar com elas. É importante destacar que não vamos nos limitar a nenhuma linguagem de programação específica, de forma que você possa entender e implementar esses TADs na linguagem que você está mais acostumado.



3. Listas

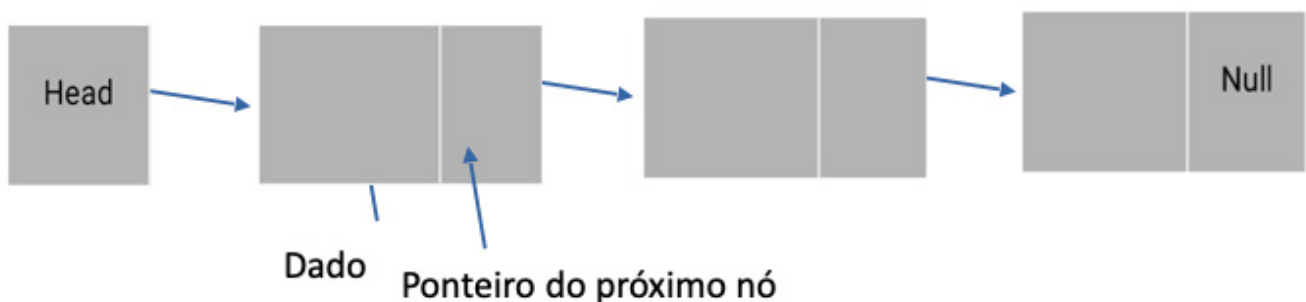
As listas são estruturas de dados simples, porém muito utilizadas. Às vezes são encontradas como sinônimos de *arrays*, mas isso depende da linguagem de programação, pois pode haver pequenas diferenças entre elas. Uma lista é uma estrutura de dados que armazena os dados em uma forma de lista, como uma lista de compras onde você tem itens. Essa lista possui vários itens, e em uma estrutura de lista podemos inserir elementos no início ou no final dela, bem como removê-los.

Conforme descrito por Pinto (2020), a lista é uma estrutura sequencial de dados que apresenta uma ordem lógica entre seus elementos, sendo que cada um desses elementos é chamado de nó ou nodo.

Nas listas, os nós são encadeados ou ligados entre si, uma vez que cada nó possui um ponteiro que aponta para o próximo elemento, ou seja,

para o nó seguinte. Esse ponteiro contém o endereço do próximo nó. No início da lista, temos o ponteiro cabeça (*head*) que aponta para o primeiro elemento da lista. A partir disso, cada nó contém um ponteiro que aponta para o próximo nó, e assim sucessivamente. Por fim, o último elemento da lista possui um ponteiro com o valor nulo ou inválido para indicar o fim da lista. A Figura 1 ilustra a estrutura de uma lista.

Figura 1 – Representação da estrutura de uma lista



Fonte: elaborada pelo autor.

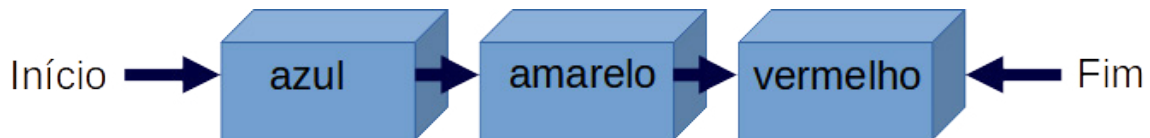
As operações mais comuns que podem ser realizadas com as listas são:

- Criação da lista.
- Inserção de elemento no início ou final da lista.
- Remoção de elemento no início ou final da lista.
- Consultar elementos na lista.
- Remoção da lista.

Conforme descrito por Pinto (2020), existem dois tipos de listas: as listas encadeadas e as listas duplamente encadeadas.

A lista encadeada é uma estrutura bem simples onde cada nó da lista faz referência ao próximo, mantendo assim uma ligação sequencial entre os elementos, como ilustrado na Figura 2.

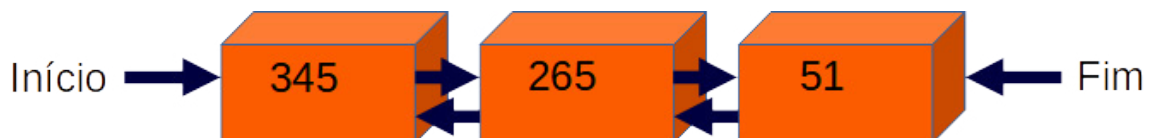
Figura 2 – Representação da estrutura de uma lista encadeada.



Fonte: elaborada pelo autor.

As listas duplamente encadeadas, como mostra a Figura 3, são como as listas encadeadas, mas são duplamente ligadas, ou seja, são bidirecionais. Nas listas encadeadas é indicado o próximo elemento, e nas listas duplamente encadeadas são indicados o próximo elemento e também o anterior. Você deve estar se perguntando, então são utilizados dois ponteiros? Sim, um indicando o próximo elemento e o outro indicando o elemento anterior.

Figura 3 – Representação da estrutura de uma lista duplamente encadeada.



Fonte: elaborada pelo autor.

Quando trabalhamos com as listas usamos a alocação dinâmica de memória, sendo assim, quando implementadas, a linguagem deve dispor de ponteiros e alocação dinâmica. Apesar dessas restrições, como a alocação é dinâmica, podemos trabalhar com tamanhos indefinidos de listas.

Agora que você já conhece as listas, vamos apresentar mais uma estrutura de dados, as pilhas.



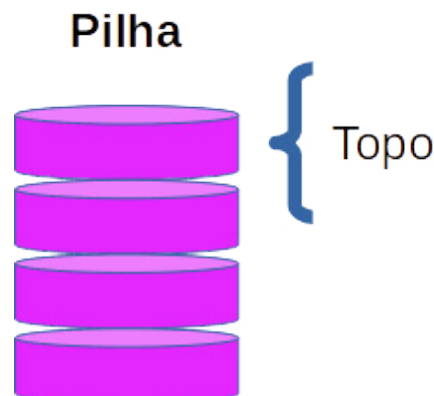
4. Pilhas

A pilha, ou também chamada de *Stack*, é uma estrutura de dados muito importante. O nome é bem sugestivo, remete-nos a uma pilha de pratos

ou livros. Mas aqui estamos falando de dados, muito dados, e você pode perguntar, a estrutura de pilha é útil? Que recurso me oferece? Imagine que você tenha uma pilha de livros e precise localizar ou retirar um livro do meio da pilha, como você faria? O mesmo ocorre com dados, vamos mostrar como podemos trabalhar com essa estrutura.

Com base em Cormen *et al.* (2002), a estrutura de dados pilha está sendo esplanada neste tópico. Para localizar um item em uma pilha que possui elementos verticalmente empilhados, é necessário remover o elemento mais recente adicionado, ou seja, o que está no topo, e continuar removendo os elementos até encontrar o item desejado. A Figura 4 ilustra uma pilha com indicação do topo, o que ajuda a compreender o funcionamento dessa estrutura.

Figura 4 – Representação da estrutura de uma pilha



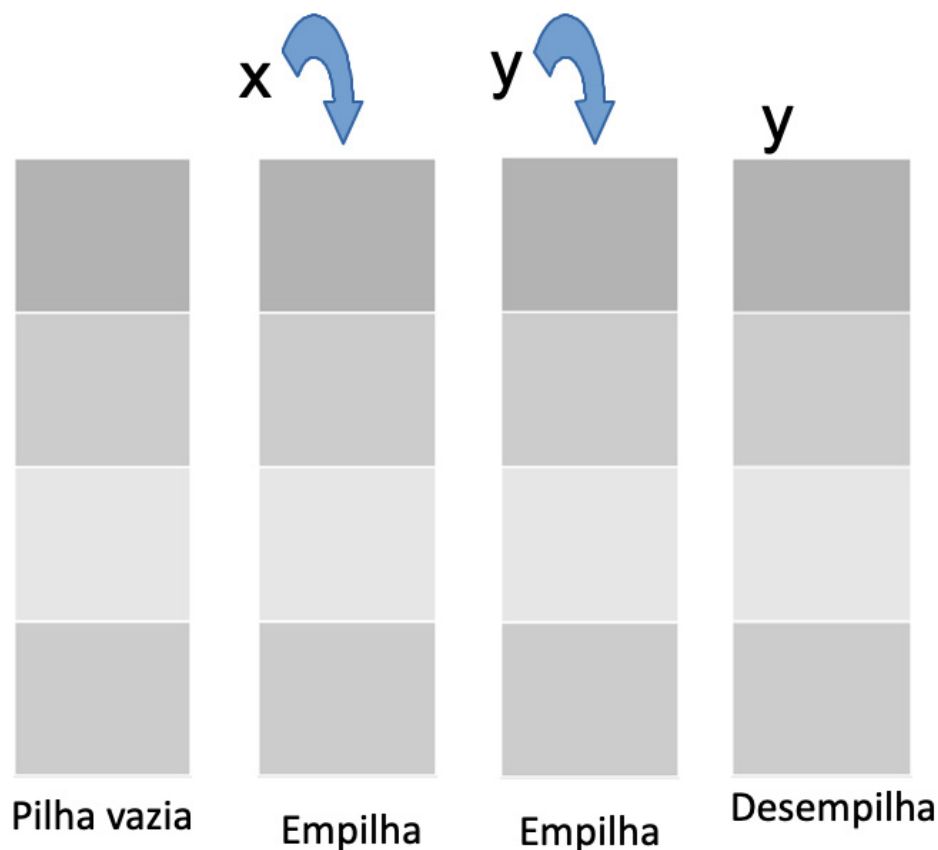
Fonte: elaborada pelo autor.

Assim funciona a pilha no processamento, sendo, como descrito, que o último elemento da pilha a entrar é o primeiro a sair. Voltando ao exemplo da pilha de livros, o último livro que foi colocado na pilha e que está no topo é o primeiro que você vai retirar para encontrar o que você deseja.

A pilha funciona de acordo com o princípio last in, first out (LIFO), no qual o último elemento adicionado é o primeiro a ser removido. No exemplo da pilha de livros, o livro mais recente adicionado ao topo da pilha é o primeiro a ser retirado para encontrar o livro desejado. Para

inserir e remover elementos na pilha, utilizamos os termos “empilhar” e “desempilhar”, respectivamente. Na Figura 5, podemos ver como os elementos são adicionados e removidos da pilha seguindo o método LIFO.

Figura 5 – Representação das operações de uma pilha



Fonte: elaborada pelo autor.

As operações com as pilhas são:

- Criar a pilha.
- Push, ou seja, o empilhar onde você insere, coloca um elemento no topo da pilha.
- Pop, é o contrário do push, aqui você desempilha, retira o elemento do topo da pilha.
- Top: observar o topo, não insere, nem retira, só verifica.

- `IsEmpty`: verifica se a pilha está vazia.
- `IsFull`: verifica se a pilha está cheia.

As pilhas são muito utilizadas para trabalhar com expressões numéricas e também quando precisamos trabalhar com recursividade.

Temos mais uma importante estrutura de dados para apresentar, na sequência você irá conhecer as filas.

5. Filas

Agora vamos conhecer as filas ou o que mais uma estrutura de dados. Como a pilha é uma estrutura de dados linear, os dados são armazenados de forma sequencial. Você pode perguntar, como funcionam as filas? Imagine que você está em um dia muito corrido tenha que passar pelo banco para algum procedimento específico. Você chega ao banco para retirar a senha e verifica que a fila está enorme, ou seja, você vai demorar para ser atendido, pois em uma fila seguimos uma ordem: o primeiro que chegar vai ser o primeiro a ser atendido.

Conforme descrito por Cormen *et al.* (2002) e Pinto (2020), no processamento, a estrutura de fila segue o princípio first in, first out (FIFO), ou seja, o primeiro elemento a entrar na fila é o primeiro a ser removido. Essa lógica é oposta à da pilha, que segue o princípio LIFO, em que o último elemento a entrar é o primeiro a ser removido. No dia a dia, quando vamos imprimir documentos costumamos dizer que temos de aguardar a fila de impressão, sim, é utilizada uma estrutura de fila no processamento para a disponibilização da impressora, sendo a primeira requisição a ser feita, vai ser o primeiro arquivo a ser liberado para a impressão. A Figura 6 mostra uma fila indicando que o elemento a ser inserido deve ser no final da fila e o elemento a ser removido deve ser o elemento no início da fila.

Figura 6 – Representação da estrutura de uma fila



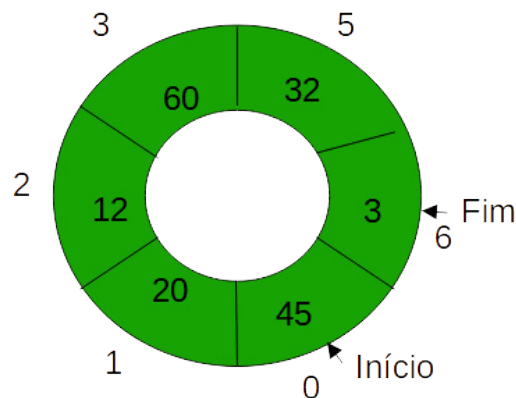
Fonte: elaborada pelo autor.

As operações da fila são:

- Criar a fila.
- Inserir elemento, ou enfileirar.
- Remover elemento ou desenfileirar.
- Consultar elemento.
- isEmpty: verifica se a fila está vazia.
- isFull: verifica se a fila está cheia.

É comum implementar as filas por arrays ou por ponteiros. Para implementar e para melhor trabalhar com a alocação de memória é utilizada a fila circular, um array como um círculo, ou seja, a primeira posição segue a última, como mostra a Figura 7. Temos aqui uma fila dinâmica, onde são alterados apenas os indicadores de início e fim. Devido a sua estrutura, otimiza-se o desperdício de espaço e isso a torna mais eficiente do que a fila linear.

Figura 7 – Representação da estrutura de uma fila circular



Fonte: elaborada pelo autor.

Na implementação por ponteiros é como nas listas, temos o elemento e o ponteiro apontando para o próximo elemento.

As filas são muito úteis quando precisamos trabalhar com compartilhamentos, como muitas pessoas aguardando liberação de acesso, sendo simples para implementar.

6. Conclusão

Foi apresentado a você o TADS, os tipos abstratos de dados, modelo matemático para trabalhar com as estruturas de dados. Vimos que os TADs incluem os dados encapsulados e as funções com operações para trabalhar sobre eles. Foram apresentadas as vantagens para trabalhar com esse modelo. Mas aprofundando um pouco foram apresentados os exemplos de TADs, as listas, pilhas e filas. As listas onde temos elementos ligados e temos a opção de inserção e remoção do primeiro e último elemento. As pilhas, onde usamos o método LIFO, o último elemento a entrar é o primeiro a sair. E as filas em que é utilizado o método FIFO, o primeiro elemento a entrar é o primeiro elemento a sair. Assim, com as particularidades de cada uma, você agora sabe como e quando podem ser utilizadas na implementação.



Referências

CORMEN, T. H. *et al.* **Algoritmos:** teoria e prática. Rio de Janeiro: Elsevier, 2002.

MANZANO, J. A. N. G. **Algoritmos:** técnicas de programação. 2. ed. São Paulo: Érica, 2015.

PINTO, R. A.; PRESTES, L. P.; SERPA, M. da S. *et al.* **Estrutura de dados.** Porto Alegre: SAGAH, 2020.

SZWARCFITER, J. L.; MARKENZON, L. **Estruturas de Dados e Seus Algoritmos.** Rio de Janeiro: LTC, 2020.

A estrutura de dados Árvores

Autoria: Virgínia Mara Cardoso

Leitura crítica: Rafael Albuquerque Pinto



Objetivos

- Apresentar os fundamentos das árvores e suas aplicações.
- Introduzir os conceitos básicos das árvores binárias e de buscas e suas operações.
- Apresentar as definições e as operações das árvores AVL s.
- Conceituar árvores B e B+ e suas operações..



1. Introdução

Nessa aula vamos conhecer as árvores, uma estrutura de dados não linear, onde cada elemento (nó) é conectado a outros elementos de forma hierárquica. A árvore é uma estrutura de dados amplamente utilizada para implementar algoritmos de pesquisa, armazenar hierarquias de informações, representar dados organizacionais, modelar estruturas de linguagem natural e muito mais.

Nesse contexto serão apresentadas as principais árvores utilizadas na programação, incluindo os conceitos de árvores binárias de busca, árvores AVL, árvores B e B+. Além disso, serão abordadas suas operações e aplicações.



2. Árvores e sua aplicações

Vamos agora conhecer a estrutura de dados árvore, que é uma estrutura de dados fundamental em computação. Quando falamos em árvore em algum estudo, logo nos referimos a uma estrutura hierárquica, ou seja, uma estrutura com ramos, ramificada, como a organização de uma empresa. Uma árvore pode ser usada para representar uma ampla variedade de informações, como a estrutura de arquivos em um sistema operacional, hierarquias organizacionais, árvores genealógicas e muito mais (CORMEN, 2002). No entanto, agora nosso foco será ver essa estrutura como uma estrutura de dados para nos ajudar a trabalhar com informações.

De acordo com Cormen (2002), as árvores são de um tipo de estruturas de dados hierárquica, composta por um conjunto de objetos interconectados chamados “nós”. Um dos nós é designado como o nó raiz, e os outros são filhos ou descendentes desse nó raiz. Cada nó pode

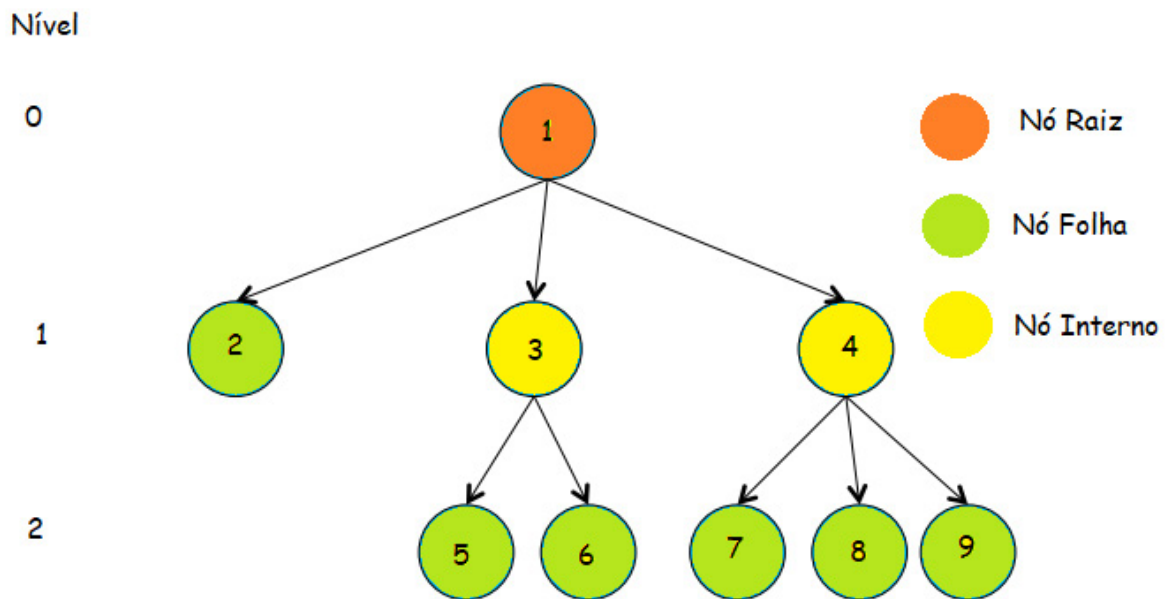
ter zero ou mais filhos, que são outros nós na árvore conectados a ele por meio de uma aresta ou ramo.

Para você se familiarizar mais facilmente com essa estrutura, temos algumas terminologias básicas associadas à árvore. De acordo com Cormen (2002), são elas:

- Nó: cada elemento na árvore.
- Raiz: é o nó no topo da árvore, que não tem pai.
- Pai: é o nó que é conectado a um nó filho, sendo o nó superior na relação hierárquica.
- Irmão: são nós que têm o mesmo pai.
- Folha: é um nó que não possui filhos, ou seja, é o nó terminal da árvore.
- Subárvore: é uma árvore dentro de uma árvore maior, formada por um nó e seus descendentes.
- Altura: é a distância do nó mais distante da raiz até a própria raiz.
- Nível ou profundidade: é a distância entre um nó e a raiz.
- Grau: é o número de filhos de um nó.

A árvore começa com um nó raiz, que é o nó superior da hierarquia. Cada nó pode ter um número variável de filhos, mas somente um pai (CELES; CERQUEIRA; RANGEL, 2016). A hierarquia é estabelecida pela relação entre o nó pai e seus filhos, como mostra a Figura 1.

Figura 1 – Representação de uma árvore



Fonte: elaborada pela autora.

As árvores podem ser classificadas conforme várias propriedades, baseado em Celes, Cerqueira e Rangel (2016) e Cormen (2002) podemos citar:

- Árvore binária: cada nó tem no máximo dois filhos.
- Árvores balanceadas: a diferença de altura entre as subárvores esquerda e direita de cada nó é no máximo um.
- Árvores enraizadas: uma árvore pode ter um ou mais nós raízes.
- Árvore de busca ou árvore de busca binária: uma árvore binária onde as chaves são organizadas de forma que as chaves menores ficam à esquerda do nó e as chaves maiores ficam à direita.
- Árvore B: uma árvore onde cada nó tem no máximo m filhos, onde m é uma constante definida pelo programador.
- Árvore B+: uma árvore B onde as chaves são armazenadas apenas nas folhas, tornando a busca mais rápida.

- Árvore AVL: uma árvore binária balanceada onde a diferença de altura entre as subárvores esquerda e direita de cada nó é no máximo um.
- Árvore rubro-negra: uma árvore binária balanceada onde cada nó é vermelho ou preto, e onde a altura negra de cada caminho da raiz a uma folha é a mesma.
- Árvore Trie: uma árvore usada para armazenar um conjunto de *strings* onde cada nó representa um prefixo comum entre as *strings*.

Você pode ter observado que temos muitas variedades de árvores e elas são utilizadas em muitas aplicações. Para escolher a árvore mais adequada para uma aplicação específica, é importante entender as propriedades e comportamentos de cada tipo de árvore. Como são estruturas de dados hierárquicas, elas são usadas para representar relacionamentos hierárquicos entre os dados. São amplamente utilizadas em sistemas de gerenciamento de banco de dados, algoritmos de busca e outras aplicações onde a hierarquia dos dados é importante.

Um ponto relevante para os programadores é que existem muitos algoritmos e bibliotecas prontas que implementam árvores e estruturas de dados relacionadas, o que facilita muito o trabalho.

Enfim, as árvores podem melhorar significativamente a performance de algoritmos que envolvem a busca, a inserção, a remoção ou a ordenação de elementos. Isso se deve ao fato de que as árvores podem reduzir drasticamente o número de operações necessárias para manipular e acessar os dados (SZWARCFITER; MARKENZON, 2020).

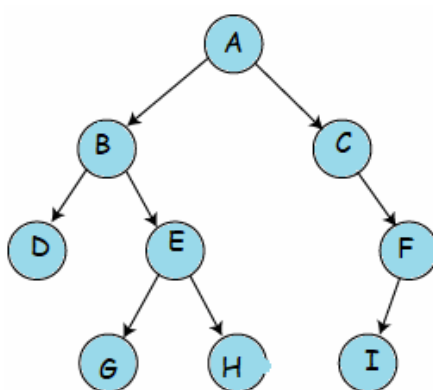


3. Árvores binárias e de busca

As árvores binárias são uma das estruturas de dados mais simples e fundamentais para a programação. Como sugere o nome, as árvores

binárias são aquelas em que cada nó pode ter no máximo dois filhos (CELES; CERQUEIRA; RANGEL, 2016). Essa propriedade permite que as árvores binárias sejam facilmente implementadas e manipuladas, tornando-as uma escolha popular para muitas aplicações de programação. Em outras palavras, as árvores binárias são estruturas de dados que consistem em um conjunto de nós interconectados, onde cada nó pode ter no máximo dois filhos, denominados subárvores esquerda e direita, conforme exemplificado na Figura 2.

Figura 2 – Representação de uma árvore binária



Fonte: elaborada pela autora.

Com as estruturas de dados árvores podemos realizar várias operações, especificamente com árvores binárias, segundo Szwarcfiter e Markenzon (2020), as principais operações são:

- **Inserção:** a inserção de um novo nó pode ser feita de diferentes maneiras, dependendo do critério utilizado para definir a posição do novo nó em relação aos nós existentes. Uma das formas mais comuns de inserção é percorrer a árvore até encontrar um nó que não tenha um filho correspondente à posição desejada e, em seguida, adicionar o novo nó como filho deste nó.
- **Busca:** a busca por um nó é realizada de forma recursiva ou iterativa. A ideia é percorrer a árvore comparando o valor procurado com o valor de cada nó visitado e, dependendo do

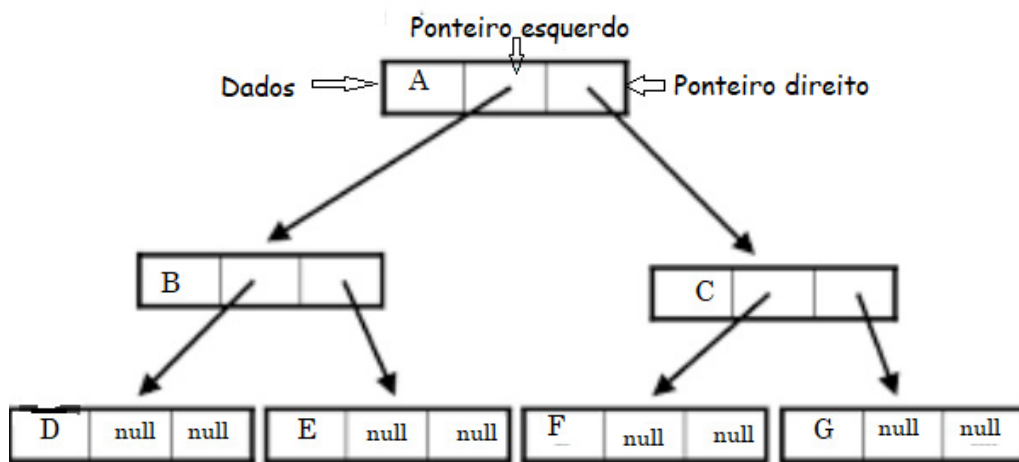
resultado da comparação, continuar a busca pela subárvore esquerda ou direita.

- **Remoção:** a remoção de um nó em uma árvore binária pode ser um pouco mais complicada do que a inserção ou a busca, pois é necessário garantir que a estrutura da árvore permaneça válida após a remoção. Existem diferentes abordagens para a remoção de um nó, mas uma das mais comuns é substituir o nó removido pelo seu sucessor (o menor nó na subárvore direita) ou pelo seu antecessor (o maior nó na subárvore esquerda).
- **Travessia:** é o processo de visitar todos os nós da árvore em uma determinada ordem. Existem três tipos principais de travessia: pré-ordem (visitar o nó raiz, em seguida, a subárvore esquerda e, finalmente, a subárvore direita), pós-ordem (visitar a subárvore esquerda, a subárvore direita e, finalmente, o nó raiz) e em ordem (visitar a subárvore esquerda, o nó raiz e, finalmente, a subárvore direita). Existem também outras formas de travessia, como a travessia em largura (ou por nível), que visita os nós em cada nível da árvore antes de passar para o próximo nível.

Segundo Celes, Cerqueira e Rangel (2016), as árvores binárias podem ser utilizadas em diversas aplicações, como em algoritmos de busca binária, em que a árvore é percorrida de acordo com determinadas condições para encontrar um determinado valor, ou em algoritmos de ordenação como o *heapsort*, em que a árvore é utilizada para construir uma estrutura de dados que permite ordenar elementos em tempo eficiente.

Quando estudamos árvores binárias temos um tipo denominado árvores de busca, ou árvores binárias de busca, que são utilizadas para armazenar dados ordenados. Cada nó da árvore contém uma chave (ou valor) e dois ponteiros, um para a subárvore esquerda (que contém chaves menores ou iguais à chave do nó) e outro para a subárvore direita (que contém chaves maiores ou iguais à chave do nó), como mostra a Figura 3. A chave em cada nó é usada para comparar com outras chaves durante a busca.

Figura 3 – Representação de uma árvore binária de busca



Fonte: elaborada pela autora.

Árvores de busca são projetadas para permitir a rápida pesquisa, inserção e exclusão de elementos. Essas árvores são amplamente utilizadas em muitas aplicações, como bancos de dados, sistemas de busca, gerenciadores de arquivos e muitos outros. A escolha da árvore de busca apropriada depende do problema em questão e das necessidades de desempenho específicas.



4. Árvores balanceadas (AVL)

Quando olhamos a estrutura das árvores às vezes ocorre das subárvores da direita e da esquerda serem de tamanhos iguais, mas não necessariamente as subárvores devem ter exatamente o mesmo tamanho. Uma árvore balanceada é uma árvore em que as subárvores esquerda e direita de cada nó possuem alturas que diferem em no máximo uma unidade. Em outras palavras, a árvore está equilibrada em torno do nó raiz.

De acordo com Cormen (2002) e Szwarcfiter e Markenzon (2020), uma árvore AVL é uma árvore binária de busca balanceada, onde a altura das subárvores esquerda e direita difere em no máximo 1. Isso garante que

a operação de busca na árvore seja eficiente, com tempo de execução $O(\log n)$, onde n é o número de nós na árvore.

Como se trata de uma árvore balanceada, para manter o balanceamento, a cada inserção ou remoção de um nó, é necessário realizar uma série de rotações na árvore para garantir que a diferença de altura entre as subárvores não exceda 1.

Baseado em Szwarcfiter e Markenzon (2020), a implementação da árvore AVL inclui operações como:

- Inserção: a inserção de um novo nó na árvore AVL pode desbalancear a árvore, o que requer uma série de rotações para reequilibrá-la.
- Remoção: a remoção de um nó também pode desbalancear a árvore, o que requer uma série de rotações para reequilibrar a árvore.
- Busca: a busca por um determinado valor é uma operação eficiente, com tempo de execução $O(\log n)$, onde n é o número de nós na árvore.
- Ordem: a árvore AVL pode ser percorrida em ordem, visitando cada nó em ordem crescente.
- Pré-ordem: percorrer em pré-ordem, quer dizer, visitar o nó raiz primeiro, seguido pela subárvore esquerda e depois pela subárvore direita.
- Pós-ordem: percorrer em pós-ordem é visitar a subárvore esquerda primeiro, seguida pela subárvore direita e depois pelo nó raiz.
- Altura: a altura da árvore pode ser calculada, o que é útil para verificar se a árvore está balanceada e para comparar a eficiência de outras estruturas de dados.

Para realizar essas operações em uma árvore AVL, é necessário implementar as funções correspondentes na linguagem de programação escolhida, seguindo as regras de rotação necessárias para manter a árvore balanceada. Essas funções podem ser implementadas recursivamente ou iterativamente, dependendo da preferência do programador.

As árvores AVL são uma estrutura de dados útil para muitas aplicações, especialmente quando a busca é uma operação crítica em termos de desempenho. No entanto, como as rotações necessárias para manter a árvore balanceada podem ser caras em termos de tempo de execução, outras estruturas de dados podem ser mais adequadas para certos tipos de problemas.

Em geral, as árvores AVL são uma estrutura de dados útil para qualquer aplicação em que as operações de busca são críticas em termos de desempenho e a estrutura de dados precisa ser mantida balanceada. Podemos citar os bancos de dados, compiladores, sistemas de gerenciamento de arquivos, e também os algoritmos de busca em jogos, como xadrez e jogo da velha. Isso permite que os algoritmos encontrem as melhores jogadas de maneira eficiente.



5. Árvores B e B+

A árvore B é uma estrutura de dados usada principalmente em bancos de dados e sistemas de arquivos. É uma árvore balanceada, que armazena um grande número de chaves e valores associados em cada nó, Cormen (2002). Baseado em Cormen (2002), uma árvore B é caracterizada por:

- Cada nó ter um número mínimo e máximo de chaves e ponteiros filhos.
- Todos os nós folha estarem no mesmo nível.

- Os dados na árvore estarem armazenados nas folhas.

Baseado em Cormen (2002), as operações mais comuns realizadas em uma árvore B são:

- Inserção: esta operação envolve a inserção de uma nova chave e o valor associado em sua posição adequada na árvore. A operação começa na raiz da árvore e é descendente até que seja encontrada uma folha adequada para inserir o novo nó. Em seguida, a árvore é atualizada para garantir que as propriedades da árvore sejam mantidas.
- Remoção: aqui temos a remoção de um nó existente. A remoção começa pela busca do nó a ser removido na árvore. Em seguida, o nó é removido e a árvore é atualizada para garantir que as propriedades da árvore sejam mantidas.
- Busca: nessa operação é preciso localizar um nó existente na árvore. A busca começa na raiz da árvore e é descendente até que o nó desejado seja encontrado ou seja determinado que o nó não está presente na árvore.
- Impressão: essa operação é para a impressão dos nós da árvore em uma determinada ordem, como ordem de percurso em ordem, pré-ordem ou pós-ordem.
- Verificação de consistência: aqui ocorre a verificação da árvore para manter todas as suas propriedades, como o número mínimo e máximo de chaves em cada nó, bem como a altura balanceada da árvore.

As árvores B e B+ são duas variações da mesma estrutura de dados em árvore, mas com algumas diferenças importantes em termos de uso e desempenho.

Uma árvore B+ é uma extensão da árvore B, na qual todos os dados estão armazenados nas folhas, enquanto os nós internos contêm apenas chaves para indexação. Isso significa que a árvore B+ é otimizada para leitura sequencial em blocos, o que a torna ideal para uso em bancos de dados ou sistemas de arquivos, onde a leitura rápida de grandes quantidades de dados é necessária.

Em geral, a árvore B é melhor para operações de escrita em disco e para pequenas quantidades de dados, enquanto a árvore B+ é melhor para leitura sequencial em blocos e para grandes quantidades de dados

6. Conclusão

As árvores são uma estrutura de dados não linear e sim hierárquica que consiste em um conjunto de nós conectados, onde cada nó tem um valor e zero ou mais filhos. Temos vários tipos de árvores classificadas com base em várias características, como número de filhos, simetria das subárvores, valores dos nós, dentre outros. Podem ser implementadas em várias linguagens de programação e são estruturas amplamente utilizadas, mas a aplicação depende muito das propriedades e comportamentos de cada tipo de árvore.

Referências

CELES, W.; CERQUEIRA, R.; RANGEL, J. **Introdução a Estruturas de Dados-Com Técnicas de Programação em C**. GEN LTC, 2016.

CORMEN, T. H. *et al.* **Algoritmos: teoria e prática**. Rio de Janeiro: Elsevier, 2002.

SZWARCFITER, J. L.; MARKENZON, L. **Estruturas de dados e seus algoritmos**. Rio de Janeiro: LTC, 2020.

Estrutura de Dados Avançadas

Autoria: Virgínia Mara Cardoso

Leitura crítica: Rafael Albuquerque Pinto



Objetivos

- Introduzir as estruturas de dados avançadas.
- Definir e explicar as Filas de Prioridade.
- Conceituar e explicar as Tabelas de Espalhamento.
- Apresentar os conceitos de Conjuntos e suas aplicações.
- Conhecer as estruturas de representação de grafos (matriz de adjacência e de incidência).



1. Introdução

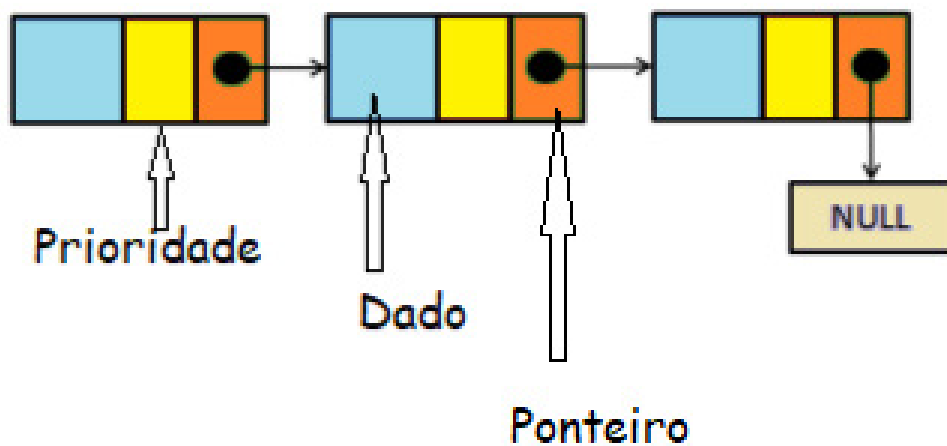
Nesta aula vamos tratar sobre as estruturas de dados avançadas. Talvez você já deva ter utilizado algumas delas. Apresentarei a você as filas de prioridade, mostrando suas operações, como e quando utilizá-las. Dentre as estruturas de dados avançadas estão também as tabelas de espalhamento, ou seja, as tabelas *hash* que nos auxiliam muito nos bancos de dados. Depois vamos ver os conjuntos, suas operações e aplicações. Para completar sobre esse assunto vamos conhecer os famosos grafos, muito utilizados no roteamento de redes.



2. Filas de prioridade

Para você entender sobre as filas de prioridade, atente-se ao nome, ou seja, trata de uma estrutura de fila, que para trabalhar com seus elementos é necessário seguir uma prioridade. São uma estrutura de dados avançada que permite armazenar um conjunto de elementos em que cada elemento é associado a uma determinada prioridade (Cormen, 2002). Segundo Szwarcfiter e Markenzon (2020), os elementos com prioridades mais altas têm prioridade sobre os elementos com prioridades mais baixas e são atendidos primeiro. A prioridade pode ser determinada por vários fatores, como tempo, custo, dentre outros, mas precisa ser ordenada. Você pode ver como é uma fila de prioridade, Figura 1, com o dado e o ponteiro para o próximo elemento como em uma fila comum, mas com uma chave.

Figura 1 – Representação de uma Fila de Prioridade



Fonte: elaborada pela autora.

Segundo Szwarcfiter e Markenzon (2020), as operações mais comuns da fila de prioridade são a inserção e remoção. Na inserção, é inserido um novo elemento na fila, de acordo com sua prioridade, e a remoção remove o elemento com a maior prioridade. Temos a consulta, o acesso ao elemento de maior prioridade. Há também a atualização da prioridade de um elemento e a união de duas filas de prioridade. São todas operações seguindo a referência da prioridade.

As filas de prioridade são comumente implementadas usando uma *heap*, que é uma estrutura de dados em árvore que possui a propriedade de que cada nó pai tem uma prioridade maior do que a de seus filhos. A *heap* pode ser uma *heap* máxima (onde a raiz tem a maior prioridade) ou uma *heap* mínima (onde a raiz tem a menor prioridade) (CORMEN, 2002).

As filas de prioridade são usadas em várias aplicações, incluindo algoritmos de busca, classificação e processamento de dados em tempo real. Por exemplo, em um algoritmo de busca, os nós do grafo são armazenados em uma fila de prioridade com base em sua distância do nó de origem. O nó com a menor distância tem a maior prioridade e é o próximo a ser explorado.

Em resumo, as filas de prioridade são uma estrutura de dados avançada que permite armazenar elementos com diferentes prioridades e são comumente implementadas usando uma *heap*.

3. Tabelas de espalhamento

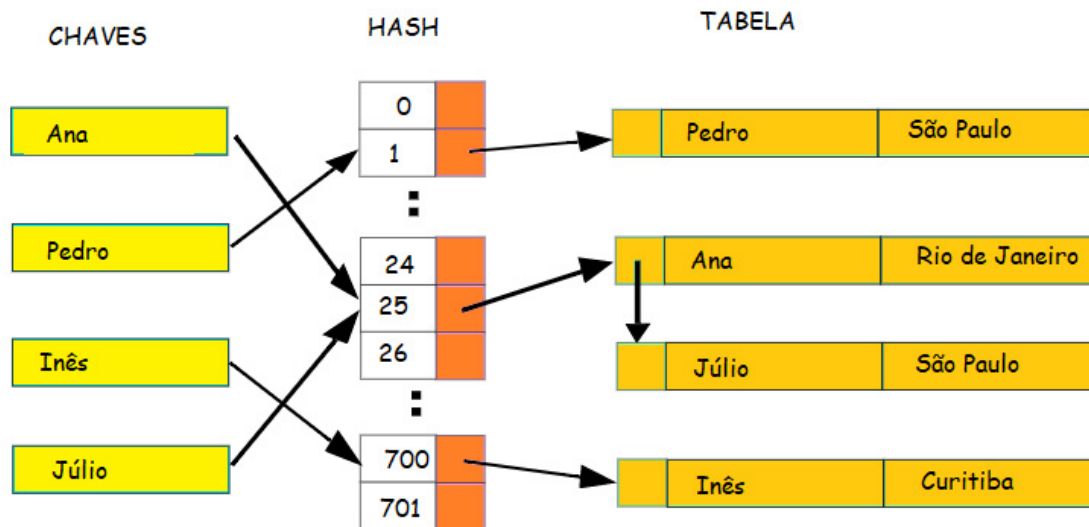
As tabelas de espalhamento são também conhecidas como tabelas *hash* ou tabelas de dispersão (CELES; CERQUEIRA; RANGEL, 2016). Se você costuma trabalhar com banco de dados, você já deve ter utilizado muito esta estrutura, pois ela é frequentemente usada para indexar e pesquisar informações. De acordo com Cormen (2002), ao invés de pesquisar uma lista de registros para encontrar um valor específico, utilizamos a tabela de espalhamento para armazenar as informações em uma estrutura de dados que permite uma pesquisa muito mais rápida. Isso ocorre porque a tabela de espalhamento usa uma função *hash* para mapear o valor de entrada para uma posição na tabela, permitindo que a pesquisa seja realizada em tempo constante.

Mas não somente em banco de dados, as tabelas de espalhamento ou *hash tables* são amplamente utilizadas em programação para armazenar e recuperar informações rapidamente.

Segundo Celes, Cerqueira e Rangel (2016), a ideia básica por trás de uma tabela de espalhamento é mapear uma chave (como uma *string* ou um número) a um valor correspondente (como um objeto ou um número) usando uma função de “espalhamento” ou “*hashing*”. Essa função calcula um índice de um vetor em que o valor correspondente àquela chave é armazenado.

Para você entender melhor, imagine um banco de dados onde as tabelas de espalhamento vão mapear as chaves (como nomes ou números) para valores associados, como mostra a Figura 2. Essas tabelas são otimizadas para permitir a rápida recuperação de valores com base em suas chaves.

Figura 2 – Representação de uma Tabela de Espalhamento



Fonte: elaborada pela autora.

A implementação de tabelas de espalhamento exige cuidado, pois é necessário lidar com colisões (quando duas chaves diferentes mapeiam para a mesma posição da tabela). Existem diversas técnicas para lidar com colisões, como a “encadeamento” (usando listas ligadas ou árvores binárias de busca para armazenar os valores que colidem), ou “re-hash” (calculando um novo índice a partir da chave e de um valor de deslocamento) (CORMEN, 2002).

Em bancos de dados, as tabelas de espalhamento são normalmente chamadas de “índices hash”, segundo Cormen (2002), e como já mencionado, são usadas para indexar dados em uma tabela, para permitir que consultas que buscam registros com um valor específico em uma coluna possam ser executadas mais rapidamente.

Por exemplo, se tivermos uma tabela de clientes com milhões de registros e quisermos localizar rapidamente uma cliente cujo nome é “Maria”, podemos criar um índice *hash* na coluna “nome”. Esse índice irá mapear cada valor da coluna “nome” a uma posição na tabela de espalhamento, e a busca pode ser executada muito mais rapidamente do que se fosse feita em todos os registros da tabela.

No entanto, é importante notar que a utilização de índices *hash* também pode ter algumas desvantagens. Por exemplo, se muitos valores mapearem para a mesma posição na tabela de espalhamento, o desempenho pode ser afetado negativamente, e o espaço necessário para armazenar o índice também pode ser significativo. Portanto, é importante avaliar cuidadosamente a necessidade e o uso de índices *hash* em um banco de dados.

A grande vantagem dessas tabelas é que a recuperação de um valor a partir de uma chave é muito rápida. Isso ocorre porque a função de espalhamento permite que a posição do valor correspondente seja encontrada de forma direta e rápida.

As tabelas de espalhamento (ou *hash tables*) possuem diversas vantagens e desvantagens, dependendo do contexto em que são utilizadas. Baseado em Cormen (2002) e Szwarcfiter e Markenzon (2020), podemos elencar algumas das principais vantagens são:

- Acesso rápido aos elementos: o acesso aos elementos em uma tabela de espalhamento é muito rápido não dependendo do tamanho da tabela. Isso a torna ideal para situações em que se precisa encontrar rapidamente um elemento específico.
- Uma implementação simples: a implementação de tabelas de espalhamento é relativamente simples em comparação com outras estruturas de dados avançadas, como árvores binárias de busca ou árvores balanceadas. Elas são compostas basicamente de um *array* de listas ligadas, o que torna a implementação e o uso dessas estruturas muito mais fáceis.

Mas, as tabelas de espalhamento, como qualquer estrutura de dados, apresenta algumas desvantagens – Cormen, (2002) e Szwarcfiter e Markenzon (2020) –, tais como:

- Possibilidade de colisões: a colisão pode afetar a eficiência das operações de busca, aumentando o tempo de execução e diminuindo a eficiência da estrutura de dados.
- Dificuldade em manter a ordem dos elementos: as tabelas de espalhamento não mantêm a ordem dos elementos, o que pode ser uma desvantagem em algumas aplicações em que a ordem é importante. Por exemplo, se precisarmos imprimir os elementos em ordem alfabética, isso pode ser mais difícil de ser feito com uma tabela de espalhamento.
- Necessidade de uma boa função de espalhamento: a eficiência de uma tabela de espalhamento depende da qualidade da função de espalhamento utilizada. Se a função de espalhamento for mal projetada, muitas colisões podem ocorrer, prejudicando o desempenho da estrutura de dados.

Podemos ver as tabelas de espalhamento como uma estrutura de dados avançada que permite o armazenamento e a recuperação rápida de informações a partir de uma chave.



4. Conjuntos

Os conjuntos são estruturas de dados avançadas que permitem armazenar um conjunto de elementos distintos sem repetição (CORMEN, 2002). Em outras palavras, um conjunto é uma coleção de elementos únicos, e não há uma ordenação, podendo ser ordenado ou não, e podem ser utilizados em diversas aplicações, como em algoritmos de processamento de linguagem natural, verificação de propriedades em sistemas formais, dentre outros.

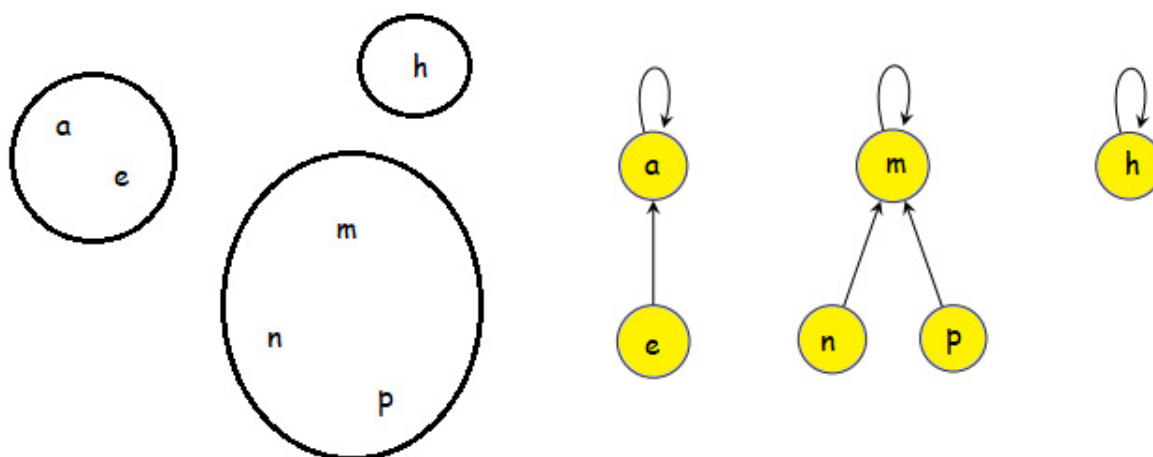
Dentre as operações mais comuns dos conjuntos são:

- Inserção de um elemento.

- Remoção de um elemento.
- Verificação da existência de um elemento no conjunto.
- União de dois conjuntos.
- Interseção de dois conjuntos.
- Diferença de dois conjuntos.

Mas como estamos falando de estrutura de dados, os conjuntos podem ser implementados por listas encadeadas, tabelas de espalhamento, árvores binárias dentre outras estruturas, como mostra a Figura 3, com a representação de conjuntos em árvores.

Figura 3 – Representação de Conjuntos em Árvores



Fonte: elaborada pela autora.

Os conjuntos possuem uma complexidade maior de implementação do que estruturas mais simples, como vetores ou listas ligadas. No entanto, a eficiência e a facilidade de uso dessas estruturas compensam esse esforço extra de implementação.

De acordo com Cormen (2002), as vantagens e desvantagens de usar conjuntos em estruturas de dados avançadas dependem da aplicação em que eles são utilizados e da implementação escolhida.

De uma forma genérica podemos citar algumas vantagens, como dizer que os conjuntos garantem que cada elemento é único, ou seja, quando você precisa armazenar uma coleção de elementos e garantir que não haja elementos duplicados. Eles permitem que você insira elementos e verifique rapidamente se eles já existem no conjunto, sem se preocupar com a possibilidade de elementos duplicados. Essas estruturas suportam operações eficientes, como calcular a união, interseção ou diferença entre dois conjuntos.

Por outro lado, os conjuntos não mantêm uma ordem definida para seus elementos. Isso pode ser uma desvantagem se você precisar acessar os elementos em uma ordem específica. Se a ordem é importante para sua aplicação, conjuntos podem não ser a melhor opção. As implementações de conjuntos geralmente têm algum *overhead* para garantir a unicidade e suportar operações eficientes. Em alguns casos, isso pode tornar os conjuntos ineficientes em termos de espaço.

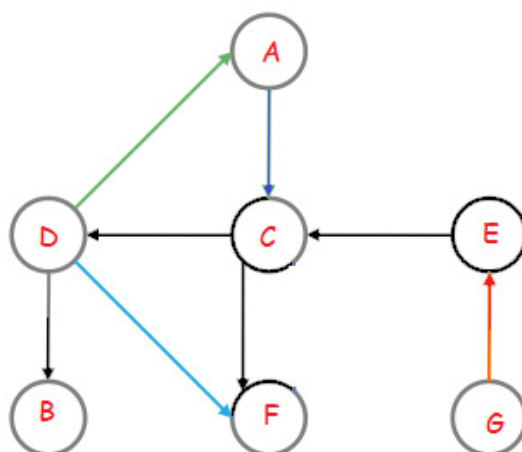
Os conjuntos têm várias aplicações em diversas áreas, como em processamento de linguagem natural, para criar dicionários de palavras e verificar se uma determinada palavra pertence a um dicionário ou não. Isso é especialmente útil em correção ortográfica, pesquisa de texto e classificação de documentos. Em sistemas de gerenciamento de bancos de dados usam conjuntos para implementar diversas operações em conjuntos de registros. Isso é particularmente útil para consultas complexas envolvendo múltiplas tabelas. Mas também podemos ver que são usados em algoritmos de aprendizado de máquina para remover dados duplicados de conjuntos de dados e para verificar se uma determinada instância de dados pertence ou não a um conjunto específico. Uma aplicação interessante de conjuntos é em criptografia, usado para criar chaves criptográficas, verificar se uma chave é válida ou não, e para remover chaves duplicadas.

Os conjuntos são estruturas de dados versáteis, eles podem ser usados em praticamente qualquer área em que haja uma necessidade de armazenar uma coleção de elementos únicos (CELES; CERQUEIRA; RANGEL, 2016).

5. Grafos

Segundo Cormen (2002), o grafo é uma estrutura matemática que consiste em um conjunto de vértices (nós) e um conjunto de arestas que conectam esses vértices. Cada aresta é uma conexão entre dois vértices e pode ter um peso ou não, como mostra a Figura 4.

Figura 4 – Representação de grafos.



Fonte: elaborada pela autora.

Existem vários tipos de grafos, como grafos direcionados e grafos não direcionados. Em um grafo direcionado, cada aresta tem uma direção, o que significa que ela vai de um vértice para outro em uma única direção. Já em um grafo não direcionado, as arestas não têm direção, o que significa que elas podem ser percorridas em ambas as direções (CELES; CERQUEIRA; RANGEL, 2016).

Além disso, os grafos podem ser ponderados ou não ponderados. Em um grafo ponderado, cada aresta tem um peso, que pode representar

uma distância, um custo, ou qualquer outra coisa. Em um grafo não ponderado, as arestas não têm peso.

As estruturas de representação de grafos são usadas para armazenar e manipular grafos em algoritmos e outras aplicações computacionais. Duas das estruturas mais comuns para representar grafos são a matriz de adjacência e a matriz de incidência.

A matriz de adjacência é uma matriz quadrada $n \times n$, onde n é o número de vértices no grafo. Cada elemento a_{ij} na matriz representa uma aresta entre o vértice i e o vértice j . Se a_{ij} for igual a 1, isso significa que há uma aresta que conecta os vértices i e j . Se a_{ij} for igual a 0, isso significa que não há aresta entre esses vértices. A matriz de adjacência é simétrica em grafos não direcionados, o que significa que a_{ij} é igual a a_{ji} .

Por outro lado, a matriz de incidência é uma matriz retangular $m \times n$, onde m é o número de arestas e n é o número de vértices no grafo. Cada linha na matriz representa uma aresta e cada coluna representa um vértice. Se a célula na linha i e na coluna j for igual a 1, isso significa que a aresta i está conectada ao vértice j . Se a célula for igual a -1, isso significa que a aresta está saindo do vértice j . Se a célula for igual a 0, isso significa que a aresta i não está conectada ao vértice j .

Segundo Cormen (2002), a matriz de adjacência é eficiente para determinar se dois vértices estão conectados por uma aresta, enquanto a matriz de incidência é eficiente para determinar as propriedades de uma aresta específica, como seus vértices adjacentes. A escolha da estrutura mais adequada para uma determinada aplicação depende do tipo de operações que serão realizadas com o grafo.

Um destaque dos grafos, é que eles são usados em uma ampla variedade de aplicações em computação, matemática, física, biologia, engenharia e muitas outras áreas.

Mas, voltado para a computação, os grafos são muito usados em algoritmos de otimização, algoritmos de caminhamento em grafos, algoritmos de roteamento em redes, algoritmos de análise de dados, entre outros.

Um dos algoritmos mais famosos que usam grafos é o algoritmo de Dijkstra, que encontra o caminho mais curto entre dois vértices em um grafo com pesos nas arestas (SZWARCFITER; MARKENZON, 2020). Aqui temos outro destaque, que é o algoritmo de PageRank do Google, que usa um grafo para classificar páginas da web com base em sua relevância.

Os grafos também são úteis na modelagem de sistemas complexos, como redes sociais, sistemas de transporte, redes elétricas e muitos outros sistemas complexos. A análise de grafos pode ajudar a entender a estrutura desses sistemas e a identificar pontos críticos e gargalos que possam ser otimizados para melhorar o desempenho ou reduzir os custos.

Resumidamente podemos dizer que os grafos são uma ferramenta poderosa para modelar e analisar sistemas complexos.



6. Conclusão

Nessa aula vimos que as estruturas de dados avançadas são essenciais para a solução de muitos problemas complexos. As filas de prioridade permitem que os elementos sejam organizados de acordo com sua prioridade, possibilitando o acesso e manipulação dos elementos mais importantes com maior eficiência. Conhecemos as tabelas de espalhamento, que permitem a busca rápida e eficiente de dados com base em suas chaves, enquanto os conjuntos fornecem uma maneira eficiente de armazenar e manipular coleções de elementos exclusivos. E finalizando, os grafos, que permitem a modelagem de relacionamentos

complexos entre objetos e entidades, possibilitando a resolução de problemas como caminho mínimo, fluxo máximo e muitos outros.

A compreensão dessas estruturas é fundamental para a criação de algoritmos eficientes e soluções para problemas complexos.



Referências

CELES, W.; CERQUEIRA, R.; RANGEL, J. **Introdução a Estruturas de Dados-Com Técnicas de Programação em C**. GEN LTC, 2016.

CORMEN, T. H. *et al.* **Algoritmos**: teoria e prática. Rio de Janeiro: Elsevier, 2002.

SZWARCFITER, J. L.; MARKENZON, L. **Estruturas de dados e seus algoritmos**. Rio de Janeiro: LTC, 2020.



BONS ESTUDOS!